10.1 Generative adversarial networks (GANs)

- in 2014, a breakthrough paper introduced *generative adversarial networks* (*GANs*), a clever new way to leverage the power of discriminative models to obtain good generative models
- at their heart, GANs rely on the idea that a data generator is good if we cannot tell fake data apart from real data
- in statistics, this is called a *two-sample test* – a test to answer the question whether datasets $X = \{x_1, \ldots, x_n\}$ and $X' = \{x'_1, \ldots, x'_n\}$ were drawn from the same distribution
- the main difference between most statistics papers and GANs is that the latter use this idea in a constructive way
- in other words, rather than just training a model to say "these two datasets do not look like they came from the same distribution", they use the two-sample test to provide training signals to a generative model
- this allows us to improve the data generator until it generates something that resembles the real data
- at the very least, it needs to fool the classifier, even if our classifier is a state-of-the-art deep neural network

10.1 Generative adversarial networks (GANs)

- the GAN architecture is illustrated in Figure 1
- as we can see, there are two parts in the GAN architecture – first off, we need a device (say, a deep network, but it really could be anything, such as a game rendering engine) that might potentially be able to generate fake data that looks just like the real data
- if we are dealing with images, this needs to generate images
- if we are dealing with speech, it needs to generate audio sequences, and so on
- we call this the *generator network*

10.1 Generative adversarial networks (GANs)

- the second component is the *discriminator network*
- it attempts to distinguish fake and real data from each other
- both networks are in competition with each other
- the generator network attempts to fool the discriminator network
- at that point, the discriminator network adapts to the new fake data
- this information, in turn, is used to improve the generator network, and so on

10.1 Generative adversarial networks (GANs)

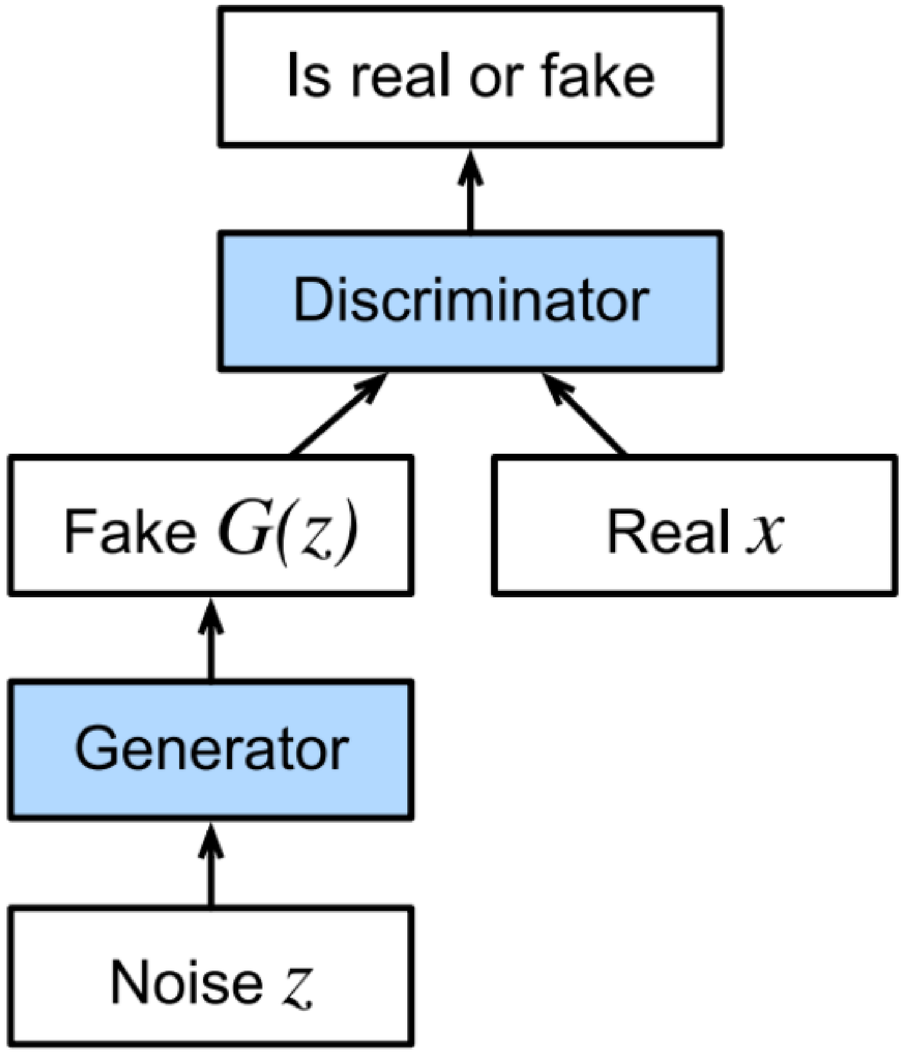

Figure 1: Generative adversarial networks (GANs).

10.1 Generative adversarial networks (GANs)

- the discriminator is a binary classifier to distinguish if the input $\boldsymbol{x}$ is real (from real data) or fake (from the generator)
- typically, the discriminator outputs a scalar prediction $o \in \mathbb{R}$ for input $\boldsymbol{x}$, such as using a fully-connected layer with hidden size 1, and then applies the sigmoid function to obtain the predicted probability $D(\boldsymbol{x}; \theta_d) = 1/(1 + e^{-o})$, where θ_d are the parameters of the discriminator
- assume the label y for the true data is 1 and for the fake data is 0
- we train the discriminator to minimize the cross-entropy loss, i.e.,

$$\min_D \{-y \log D(\boldsymbol{x}) - (1 - y) \log(1 - D(\boldsymbol{x}))\}.$$

- the generator first draws some parameter $\boldsymbol{z} \in \mathbb{R}^d$ from a source of randomness, e.g., a normal distribution $\boldsymbol{z} \sim \mathcal{N}(0, 1)$
- we often call $\boldsymbol{z}$ the *latent variable*

10.1 Generative adversarial networks (GANs)

- it then applies a function $G(\boldsymbol{z}; \theta_g)$, where θ_g are the parameters of the generator, to generate $\boldsymbol{x}' = G(\boldsymbol{z})$
- the goal of the generator is to fool the discriminator to classify $\boldsymbol{x}' = G(\boldsymbol{z})$ as true data, i.e., we want $D(G(\boldsymbol{z})) \approx 1$
- in other words, for a given discriminator D, we update the parameters of the generator G to maximize the cross-entropy loss when $y = 0$, i.e.,

$$\max_G \{-(1-y)\log(1 - D(G(\boldsymbol{z})))\} = \max_G \{-\log(1 - D(G(\boldsymbol{z})))\}.$$

- if the generator does a perfect job, then $D(\boldsymbol{x}') \approx 1$, so the above loss is near 0, which results in the gradients that are too small to make good progress for the discriminator

10.1 Generative adversarial networks (GANs)

- so, commonly, we minimize the following loss:

$$\min_G\{-y\log(D(G(\boldsymbol{z})))\} = \min_G\{-\log(D(G(\boldsymbol{z})))\},$$

  which is just feeding $\boldsymbol{x}' = G(\boldsymbol{z})$ into the discriminator, but giving label $y = 1$
- to sum up, D and G are playing a "minimax" game with the comprehensive objective function:

$$\min_D \max_G\{-E_{\boldsymbol{x}\sim\text{Data}}[\log D(\boldsymbol{x})] - E_{\boldsymbol{z}\sim\text{Noise}}[\log(1 - D(G(\boldsymbol{z})))]\}.$$

- many of the applications of GANs are in the context of images
- the stochastic gradient descent training of GANs follows (k is a hyperparameter used with value $k = 1$ in the experiments):

10.1 Generative adversarial networks (GANs)

for number of training iterations **do**
  for k steps **do**
    Sample minibatch of m noise samples $\{\boldsymbol{z}^{(1)}, \ldots, \boldsymbol{z}^{(m)}\}$ from Noise.
    Sample minibatch of m examples $\{\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(m)}\}$ from data generating distribution Data.
    Update the discriminator by stochastic gradient descent:

$$\theta_d \leftarrow \theta_d - \eta \nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^{m} \left[-\log D\left(\boldsymbol{x}^{(i)}\right) - \log\left(1 - D\left(G\left(\boldsymbol{z}^{(i)}\right)\right)\right)\right].$$

  end for
  Sample minibatch of m noise samples $\{\boldsymbol{z}^{(1)}, \ldots, \boldsymbol{z}^{(m)}\}$ from Noise.
  Update the generator by stochastic gradient ascent:

$$\theta_g \leftarrow \theta_g + \eta \nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^{m} -\log\left(1 - D\left(G\left(\boldsymbol{z}^{(i)}\right)\right)\right).$$

end for